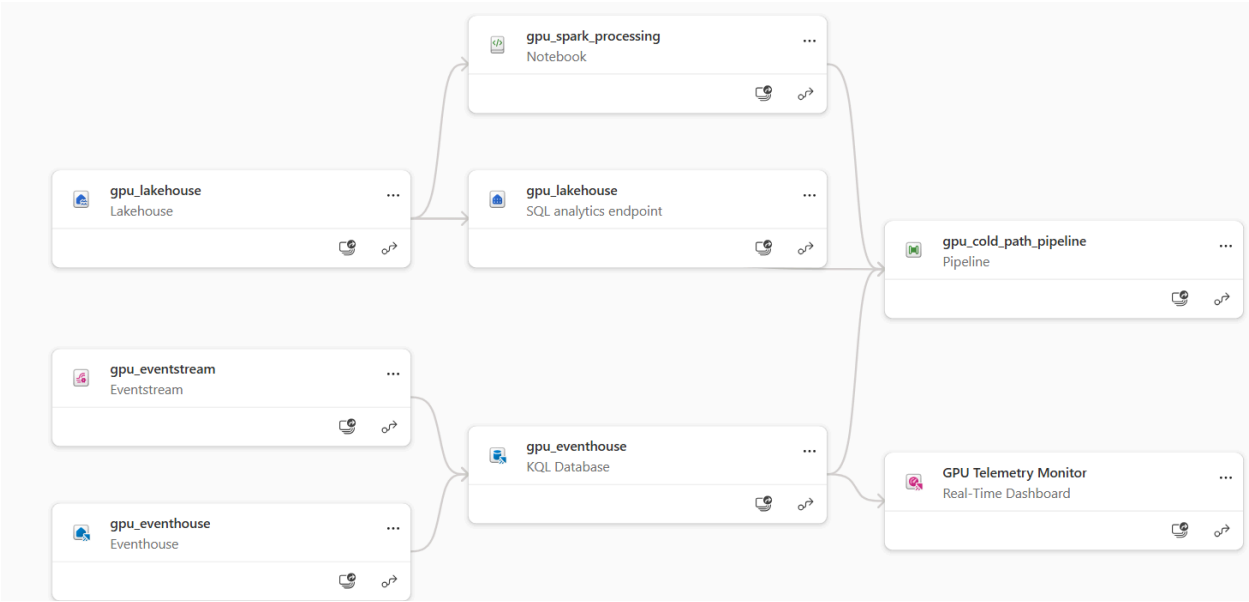
Unified GPU Telemetry

Automation Documentation

A data pipeline for ingesting and processing GPU telemetry on Microsoft Fabric

Item	Detail
Platform	Microsoft Fabric (Eventstream, Eventhouse, Fabric Data Pipeline , Lakehouse)
Scope	Automation: how runs start, how data flows, how failures are handled, how success is verified
Paths	Hot Path (real-time monitoring) and Cold Path (scheduled analytics preparation)



1. Overview

This document covers the **Automation** side of the project: how processing starts, how data flows through the system, how the system reacts to failure, and how we confirm that a run succeeded.

1.1 Summary

GPU telemetry readings are sent by a Python Producer into Fabric. A **Hot Path** cleans and displays them within seconds. A **Cold Path** runs on a schedule and moves the data into a Lakehouse as structured Silver and Gold tables for historical analysis. Both paths are built to survive failures without losing data and to be safe to re-run without creating duplicates.

1.2 Data disclaimer

> **Note on the data**
> H100 and B200 data is based on selected source traces. A100, L40S and MI300X data is **simulated** using assumed scaling factors. Some traces are repeated to extend the timeline, and repetition does not represent new real experiments. The goal of the project is to build a data pipeline and test dashboards, **not** to prove that one GPU type outperforms another.

1.3 Implementation status

Component	Status
Python Producer (Retry, Checkpoint, Dead-letter, Logging)	Implemented
Eventstream, then Eventhouse (Raw / Cleaned / Quarantine)	Implemented
Real-Time Dashboard	Implemented
Scheduled Fabric Data Pipeline(Lookup, Copy, Notebook, Log)	Implemented
Lakehouse (Silver / Gold) using PySpark	Implemented

1.4 Design principles

- **Do not lose data.** Retries, a dead-letter file, and a watermark that moves only after success.
- **Be safe to repeat.** MERGE on `event_id` and de-duplication make retries and re-runs harmless.
- **Be observable.** Logs, a freshness indicator, a quarantine table, and a watermark log table show what happened.
- **Separate speed from structure.** The Hot Path is optimized for latency, the Cold Path for clean historical data.

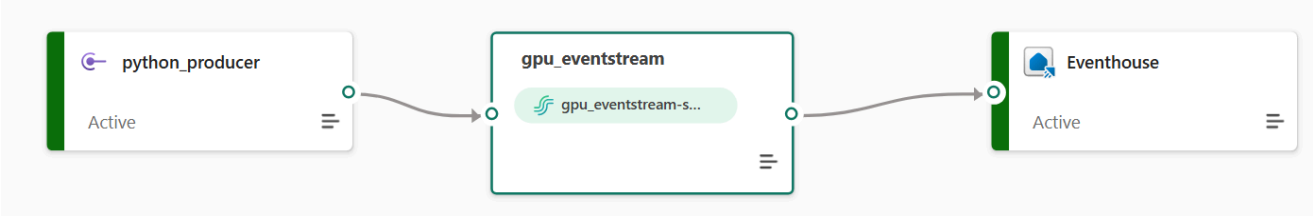
2. The Four Automation Concepts

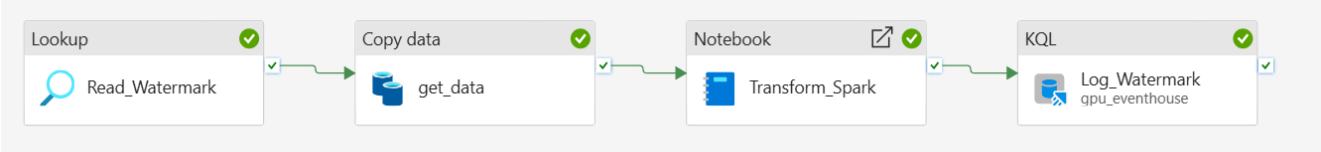
Concept	Hot Path (real time)	Cold Path (scheduled)
Trigger	The Producer runs continuously; no per-record trigger is needed	Schedule trigger on the Pipeline (hourly)
Automation	Eventstream and Update Policy run automatically	Lookup, then Copy, then Notebook, then Log, in a fixed automatic order
Retry	Inside the Producer (exponential backoff)	Retry setting on every Activity
Monitoring	Producer log, Data Freshness, Quarantine table	Monitor hub and the `gpu_watermark` table

Why two paths? The Hot Path gives immediate visibility into what is happening. The Cold Path prepares organized, de-duplicated data for historical analysis. Each is tuned for a different need.

3. Architecture

```
Telemetry CSV
   |
Python Producer   (Retry, Checkpoint, Dead-letter, Logging)
   | JSON
Fabric Eventstream
   |
   Eventhouse (gpu_raw > Update Policy > gpu_cleaned / gpu_quarantine)
      |
      +--> Real-Time Dashboard                  (Hot Path)
      |
      +--> Fabric Data Pipeline (Schedule)      (Cold Path)
              Lookup > Copy > Notebook > Log
                       |
              Lakehouse: Silver > Gold
```





4. Hot Path Automation

4.1 Producer and Retry

- Reads the CSV and sends a batch at a fixed interval. A batch of 5 records equals one server reading per GPU type for one minute.
- **Retry:** up to 5 attempts with exponential backoff (1, 2, 4, 8, 16 seconds, capped at 30) plus random jitter so retries do not synchronize.
- **Logging:** `producer.log` plus counters: `sent`, `failed`, `retries`, `bad_injected`, `dead_lettered`.
- **Type conversion before sending:** numbers are sent as numbers, timestamps in ISO format, and empty values become null (for example coolant in L40S, which is air-cooled).
- **Bad-record injection for testing** (`--bad-rate`): temperature of 900, utilization of 180, a missing value, or an invalid timestamp. Their IDs start with `BAD_` so they are easy to identify.

```
python producer.py --file telemetry.csv --interval 10 --batch-size 5 --limit 1800 --bad-rate
0.05
```

4.2 Checkpoint

Goal: after a restart, the Producer should not resend what was already sent. This saves time and reduces duplicates.

State is stored in `checkpoint.json`:

```
{"file": "<full path of the CSV file>", "next_index": 20}
```

This means the first 20 rows were sent and the next run starts from row 21.

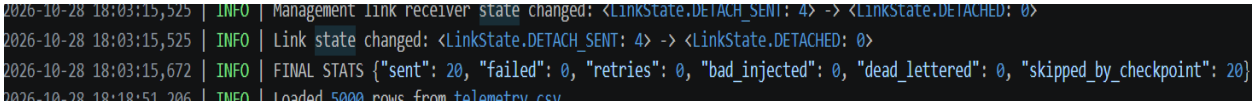
Rules

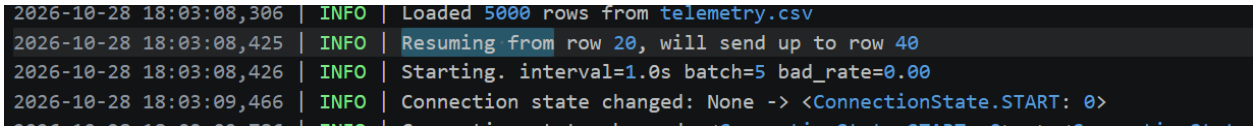
- Updated after each batch, **after** `send_with_retry` finishes, meaning after success or after the batch moves to the dead-letter file. It is not updated during retry attempts.
- **Atomic writes:** written to a temporary file, then swapped in, so the file is never corrupted if the machine stops mid-write.
- If the CSV path differs from the saved one, the checkpoint is ignored.
- `--reset` deletes the checkpoint and starts from row 1.
- `--limit` is the maximum number of rows in **this** run, counted from the checkpoint.

Why does the checkpoint advance even when a batch goes to dead-letter? Because those records are safely stored there and are resent by a separate script. Rolling the checkpoint back would resend everything after that point and create duplicates.

Known limit
If the Producer stops after a batch arrives but before the checkpoint is saved, at most **one batch** is resent. A second line of defense handles this: de-duplication by `event_id` in the Dashboard and in Silver.

Evidence from the log: after stopping and restarting, the log showed `Resuming from row 20` and `skipped_by_checkpoint: 20`.





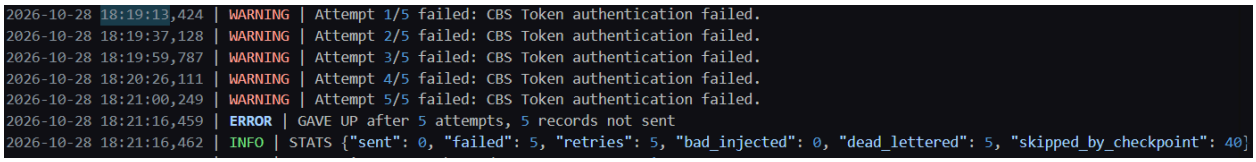


4.3 Dead-letter and Replay

- After all retries fail, the Producer writes the records to `dead_letter.jsonl` together with the failure reason, then **keeps running** instead of stopping.
- `replay_dead_letter.py` renames the file to `dead_letter.replaying.jsonl` and sends it using the same retry logic. Records that fail again go to a new `dead_letter.jsonl`; on success the temporary file is deleted. If it stops midway, it resumes from the temporary file (one batch may repeat).
- The script never touches the checkpoint.
- **Handling late data:** replayed records keep an old `ts` but get a new `ingestion_time`. Because the Cold Path watermark is based on `ingestion_time`, it picks them up instead of missing them.

How it was tested: the Connection String key was temporarily replaced with a wrong one. The log showed failed attempts followed by `GAVE UP`, and the dead-letter file filled up. After restoring the correct key we ran:

```
python replay_dead_letter.py
```



dead_letter.jsonl


```
{"reason": "CBS Token authentication failed.\nStatus code: None\nError: client-error\nCBS Token authentication failed.\nStatus code: None", "record": {"event_id": "EVT_0
{"reason": "CBS Token authentication failed.\nStatus code: None\nError: client-error\nCBS Token authentication failed.\nStatus code: None", "record": {"event_id": "EVT_0
{"reason": "CBS Token authentication failed.\nStatus code: None\nError: client-error\nCBS Token authentication failed.\nStatus code: None", "record": {"event_id": "EVT_0
{"reason": "CBS Token authentication failed.\nStatus code: None\nError: client-error\nCBS Token authentication failed.\nStatus code: None", "record": {"event_id": "EVT_0
{"reason": "CBS Token authentication failed.\nStatus code: None\nError: client-error\nCBS Token authentication failed.\nStatus code: None", "record": {"event_id": "EVT_0
```


4.4 Real-Time Cleaning (Update Policy)

Every record that reaches `gpu_raw` is automatically routed by an Update Policy into two tables:

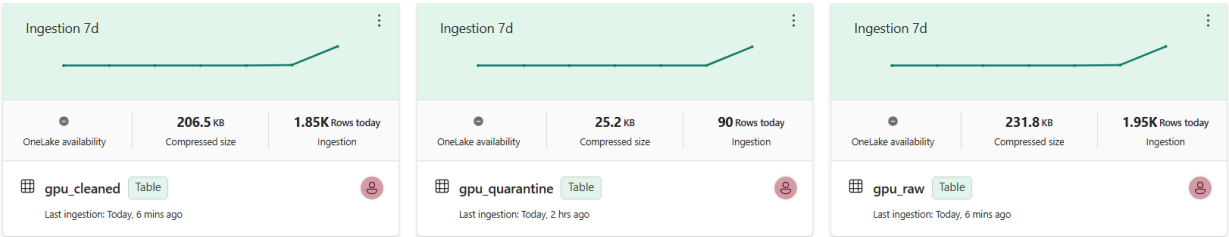
- `gpu_cleaned` via `fn_gpu_clean()`: valid records with correct data types.
- `gpu_quarantine` via `fn_gpu_quarantine()`: rejected records with a `reject_reason` and the original `raw_record`.

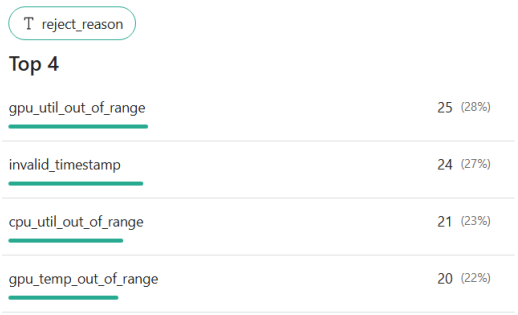
Rule	Allowed range
GPU and CPU utilization	0 to 100
GPU and CPU temperature	0 to 120
GPU power	0 to 20,000 W
Humidity	0 to 100
Memory used	Less than or equal to total memory
event_id, ts, server_id	Must be present

Empty coolant values in air-cooled systems do **not** cause rejection. The limits are approximate and should be reviewed by a domain expert.

Failure behavior

The Update Policy is set to `IsTransactional: false`. If it fails, the record stays in `gpu_raw` with no visible error. For this reason we rely on the count reconciliation check in section 7.





4.5 Real-Time Monitoring (Dashboard)

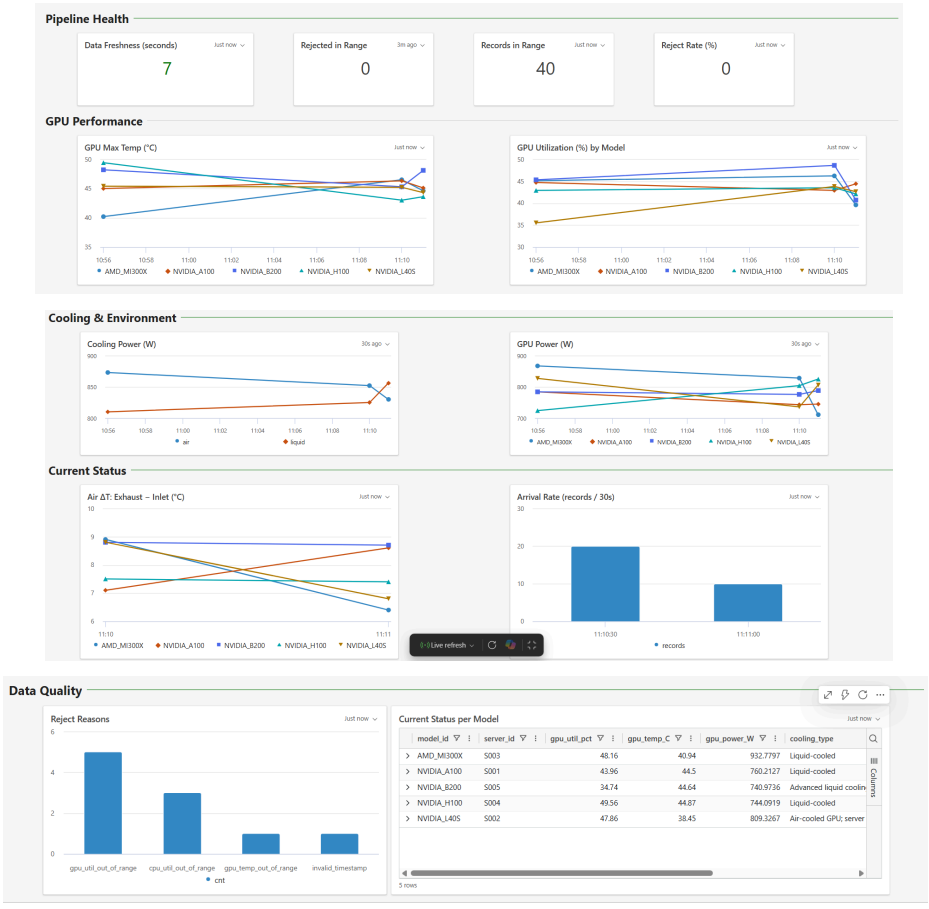
- The time filter uses `ingestion_time()` rather than `ts`. `ts` comes from the file and is old, so any recent time filter on it would show nothing.
- A base query named `live` reads `gpu_cleaned` and removes duplicates by `event_id`; a second one, `live_bad`, reads rejected records.

```
gpu_cleaned
| where ingestion_time() between (_startTime .. _endTime)
| extend ing = ingestion_time()
| summarize arg_max(ing, *) by event_id
```

- **Data Freshness** is the health indicator: seconds since the last record. If the number grows, sending has stopped.

```
gpu_cleaned
| summarize last_ingest = max(ingestion_time())
| project seconds_since_last = datetime_diff('second', now(), last_ingest)
```

- Other tiles: valid vs rejected counts and rejection rate, GPU utilization / temperature / power per type, cooling, a current-status table, arrival rate, and rejection reasons



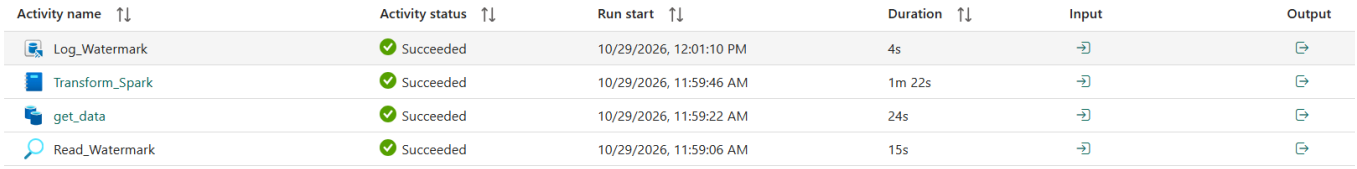
5. Cold Path Automation

5.1 The Pipeline

The pipeline is named `gpu_cold_path_pipeline` and runs these Activities in order:

#	Activity	Type	Purpose
1	`Read_Watermark`	Lookup (KQL)	Reads the last `window_end` and computes the new window
2	`get_data`	Copy data	Copies `gpu_cleaned` rows for the window into the Lakehouse
3	`Transform_Spark`	Notebook (`gpu_spark_processing`)	Cleans, de-duplicates, MERGEs into Silver, then builds Gold
4	`Log_Watermark`	KQL	Records the window and row count after everything before it succeeded

The order is intentional. The watermark is updated only after copy and transform succeed. If any step fails, the watermark does not move, so the next run repeats the same window and no data is lost.



Activity name	Activity status	Run start	Duration	Input	Output
Log_Watermark	Succeeded	10/29/2026, 12:01:10 PM	4s		
Transform_Spark	Succeeded	10/29/2026, 11:59:46 AM	1m 22s		
get_data	Succeeded	10/29/2026, 11:59:22 AM	24s		
Read_Watermark	Succeeded	10/29/2026, 11:59:06 AM	15s		

5.2 The Watermark

The `gpu_watermark` table in the Eventhouse has two roles: it defines the next window, and it is a monitoring log for every run.

```
.create table gpu_watermark (run_id:string, window_start:datetime, window_end:datetime,
rows_copied:long, status:string, logged_at:datetime)
```

It starts with one `INIT` row whose `window_end` is 2000-01-01. The Lookup query is:

```
let wm = toscalar(gpu_watermark | where status in ('INIT','OK') | summarize max(window_end));
print window_start = wm, window_end = now() - 1m
```

- The transferred window is `(window_start, window_end]`. `window_end` is the current time minus one minute, a safety margin so a record being ingested at that moment is not missed.
- Filtering uses `ingestion_time`, not `ts`, so late data (such as records replayed from dead-letter) is captured.
- `Log_Watermark` takes the window and the row count from the outputs of earlier Activities through dynamic content. It does **not** call `now()` itself, because a different window between Copy and Log would open the door to lost or duplicated data.

```
@concat('.set-or-append gpu_watermark <| print run_id="', pipeline().RunId,
  '", window_start=datetime("', activity('Read_Watermark').output.firstRow.window_start,
```

```
'"), window_end=datetime("', activity('Read_Watermark').output.firstRow.window_end,
'"), rows_copied=long(', activity('get_data').output.rowsRead,
'), status="OK", logged_at=now()')
```

> **Implementation detail**
> The Copy activity in this environment exposes **rowsRead** , not `rowsCopied`. Using `rowsCopied` fails with "property doesn't exist". `rowsRead` equals the rows written when the source is filtered by the window.

Note: the first two rows in the table were recorded before this fix, with fixed values (`window_start = 2000-01-01`, `rows_copied = 45`). They do not affect runs and are excluded from the verification query.

Table 1 | Search | 2026-10-29 11:19 (UTC)

logged_at	window_start	window_end	rows_copied	gap_min
2026-10-28 22:15:25.2606063	2026-10-28 20:45:59.7143672	2026-10-28 22:12:35.5754331	0	
2026-10-29 07:42:23.4514356	2026-10-28 22:12:35.5754331	2026-10-29 07:39:25.5285486	15	0
2026-10-29 08:41:58.6706195	2026-10-29 07:39:25.5285486	2026-10-29 08:39:22.2683441	1,235	0
2026-10-29 09:01:12.9347079	2026-10-29 08:39:22.2683441	2026-10-29 08:58:19.3470803	390	0
2026-10-29 09:36:07.0164949	2026-10-29 08:58:19.3470803	2026-10-29 09:33:21.4738254	215	0

5.3 The Notebook (PySpark)

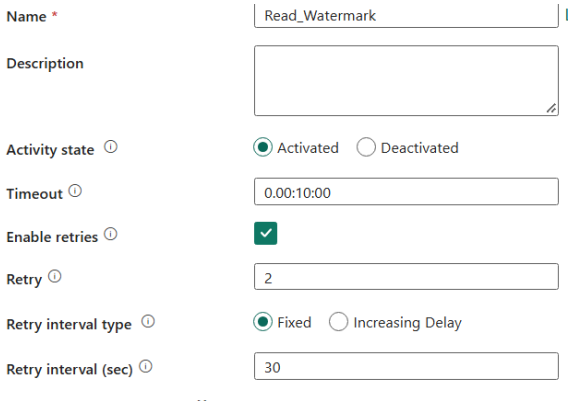
- Reads `gpu_cleaned` from the Lakehouse, drops rows missing `event_id`, `ts` or `model_id`, and removes duplicates by `event_id`.
- Adds derived columns: `date`, `hour_start`, `gpu_mem_used_pct`, `gpu_power_per_gpu_W`, `air_delta_T_C`, `cooling_to_gpu_power_ratio`, `cooling_category`.
- **Silver** (`dbo.silver_gpu_telemetry`): MERGE on `event_id`, partitioned by `date`. Re-running does not duplicate rows (idempotent).
- **Gold** (`dbo.gold_gpu_hourly`): averages and maximums per `model_id` per hour. It recomputes affected hours from the whole Silver table and then MERGEs, so late data does not corrupt the numbers.
- At the end it computes a reconciliation and returns it to the Pipeline: `silver_duplicates` and `batch_missing_in_silver` must be zero and `status = OK`.

> **Known limit**
> The Lakehouse source table has no `ingest_time` column, so the Notebook processes the **whole table** on every run. This is safe because of MERGE. To make it fully incremental, add that column during the Copy step in future.

5.4 Retry and Timeout

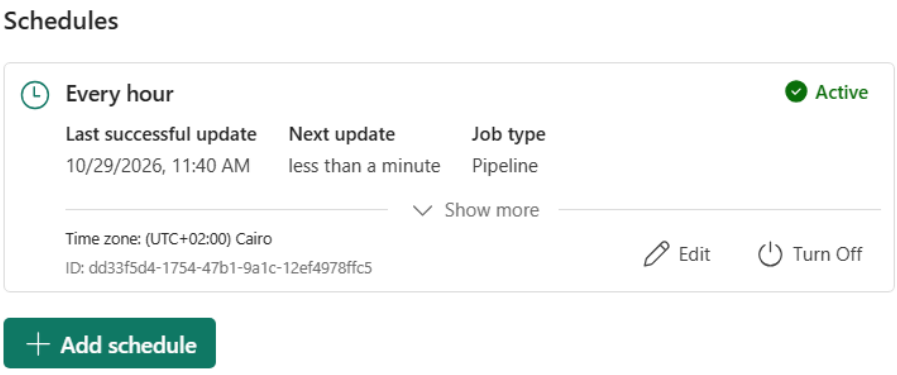
Activity	Retry	Retry interval	Timeout
`Read_Watermark`	2	30 seconds	10 minutes
`get_data`	3	60 seconds	15 minutes
`Transform_Spark`	2	60 seconds	30 minutes
`Log_Watermark`	3	30 seconds	10 minutes

Retry is safe on the Notebook because MERGE prevents duplicates on re-execution. This retry covers Cold Path failures, while the Producer retry covers sending failures. Explicit timeouts replace the 12-hour default, so a hung run cannot block the next one.



5.5 Trigger (Schedule)

- Schedule type **Fixed**, repeat **Hourly**, interval 1 hour, time zone **(UTC+02:00) Cairo**.
- A defined end date rather than open-ended, because the trial has limited duration.
- The interval is much longer than a run, which prevents overlapping runs and eases pressure on Spark capacity.
- Manual runs are always available with the Run button.
- **Recommended:** set pipeline **Concurrency = 1** so two runs can never read the same watermark.



5.6 Failure Scenarios

How the system behaves when something goes wrong, and how to recover:

Failure	Effect	Detection	Recovery
Network drop while sending	Batch delayed	Producer log: Attempt n/5 failed	Automatic retry with backoff
All send retries fail	Batch goes to dead-letter	GAVE UP in log; `dead_letter.jsonl` not empty	Run `replay_dead_letter.py`
Invalid record	Rejected from Cleaned	Row in `gpu_quarantine` with reason	Inspect reason; fix source if needed
Update Policy fails	Record stays in Raw only	Reconciliation `missing` greater than 0	Fix function and re-ingest
Copy or Notebook fails	Watermark does not move	Failed run in Monitor; no new OK row	Automatic retry, then next run repeats the window
Log_Watermark fails after data was copied	Same window is processed again	Gap or repeated window in watermark query	Safe: MERGE prevents duplicates
Producer stopped	No new data	Data Freshness rises; Arrival Rate falls to zero	Restart; checkpoint resumes

6. Delivery Guarantees and Duplicate Handling

The system provides **at-least-once** delivery and gets to an effectively exactly-once result through de-duplication at several layers:

Layer	Mechanism	Protects against
Producer	Checkpoint (atomic writes)	Resending after a restart
Dashboard	`arg_max` by `event_id`	Duplicates from old `--reset` runs or one repeated batch
Cold Path window	Watermark moves only on success	Lost data after a failure
Notebook input	Drop duplicates by `event_id`	Duplicates inside a batch
Silver	MERGE on `event_id`	Duplicates across runs and retries
Gold	Recompute affected hours, then MERGE	Wrong aggregates from late data

7. Verification Queries

7.1 Hot Path

A. Count reconciliation. No records should be lost between Raw, Cleaned and Quarantine.

```
let a = ago(30m); let b = ago(2m);
let raw = toscalar(gpu_raw | where ingestion_time() between (a .. b) | count);
let good = toscalar(gpu_cleaned | where ingestion_time() between (a .. b) | count);
let bad = toscalar(gpu_quarantine | where ingestion_time() between (a .. b) | count);
print raw, good, bad, missing = raw - good - bad
```

Expected: `missing = 0`. The last two minutes are excluded so the Update Policy has time to finish



Table 1

print_0	print_1	print_2	missing
76	60	16	0

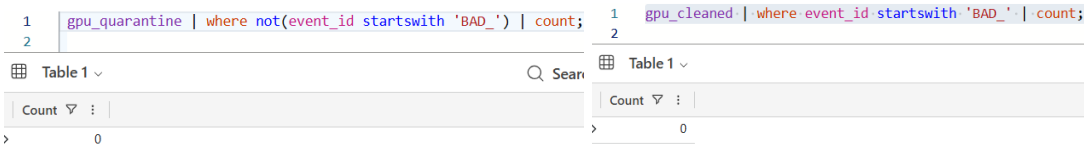
B. Bad-record separation.

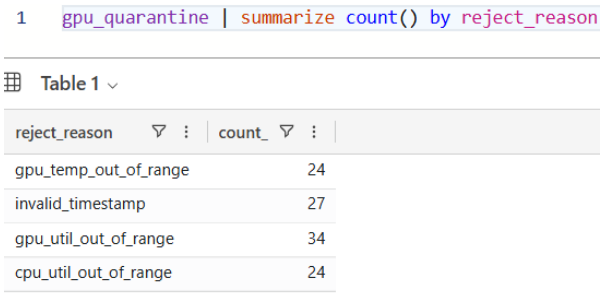
```
gpu_cleaned | where event_id startswith 'BAD_' | count

gpu_quarantine | where not(event_id startswith 'BAD_') | count

gpu_quarantine | summarize count() by reject_reason
```

Expected: the first two return zero. The third shows the rejection reasons.





reject_reason	count_
gpu_temp_out_of_range	24
invalid_timestamp	27
gpu_util_out_of_range	34
cpu_util_out_of_range	24

C. Schema and mapping.

```
gpu_raw | getschema | where ColumnName matches regex '_[0-9]$'

.show table gpu_raw ingestion mappings
```

Expected: no results from the first query, and no mapping that points to columns ending in _1.

```
gpu_raw | getschema | where ColumnName matches regex '_[0-9]$'
```

Table 1

ColumnName	ColumnOrdinal	DataType	ColumnType

```
.show table gpu_raw ingestion mappings
```

Table_0 — Search — 2026-10-29 11:50 (UTC) — # 1 — Hide empty columns

Name	Kind	Mapping	LastUpdatedOn	Database	Table
gpu_raw_mapping	Json	[{"column":"event_id","path":"$['event_id']","datatype":""},{"column":"	2026-10-28 09:29:23.3636302	e83352bf-590f-4b94-aca5-8f19f7113bc6	gpu_raw

D. Ingestion failures and policy.

```
.show ingestion failures | where FailedOn > ago(1h)

.show table gpu_cleaned policy update
```

Expected: no failures, and `IsEnabled = true`.

```
.show ingestion failures | where FailedOn > ago(1h)
```

Table_0

OperationId	Database	Table	FailedOn	IngestionSourcePath	Details	FailureKind	Roo

```
.show table gpu_cleaned policy update
```

Table_0 — Search — 2026-10-29 11:52 (UTC) — #

PolicyName	EntityName	Policy
UpdatePolicy	[e83352bf-590f-4b94-aca5-8f19f7113bc6].[gpu_cleaned]	[{ "IsEnabled": true, "Source": "gpu_raw", "Query": "fn_gpu_clean()",

7.2 Producer Tests

Test	Steps	Expected
Checkpoint	Run with `--limit 10`, then again without `--reset`	`gpu_raw` grows by 10 each time, with no repeated rows
Retry	Disconnect the internet for about 20 seconds while sending	`Attempt n/5 failed`, then `Succeeded after n attempts`
Dead-letter	Use a Connection String with a wrong key	`GAVE UP` appears and `dead_letter.jsonl` fills
Replay	Run `python replay_dead_letter.py`	The file disappears and `sent` equals the record count
Bad records	Run with `--bad-rate 0.1`	All `BAD_` records are in Quarantine

7.3 Cold Path

A. Watermark is continuous with no gaps.

```
gpu_watermark
| where status == 'OK' and window_start > datetime(2001-01-01)
| order by window_start asc
| extend prev_end = prev(window_end)
| project logged_at, window_start, window_end, rows_copied,
          gap_min = datetime_diff('minute', window_start, prev_end)
```

Expected: `gap_min = 0` (the first row is empty). In our runs the gap was 0 and `rows_copied` held real numbers (for example 1235 and 390), and zero when the Producer was stopped, without the run failing.

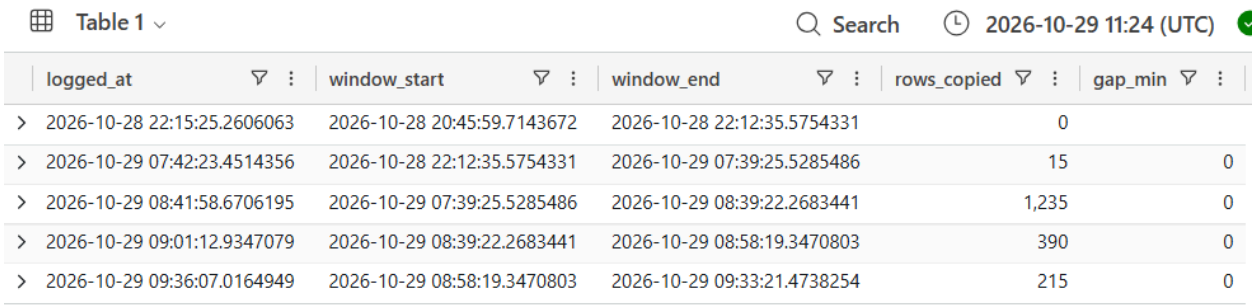


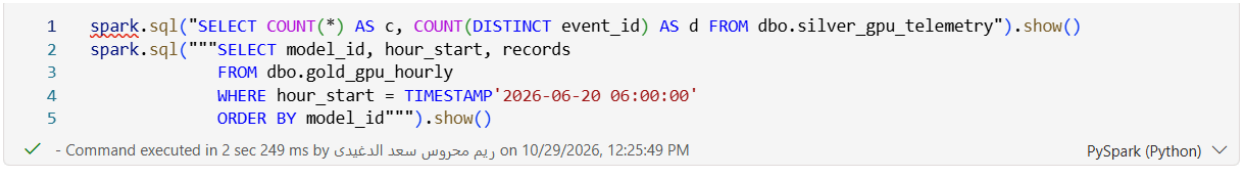
Table 1 — Search — 2026-10-29 11:24 (UTC)

logged_at	window_start	window_end	rows_copied	gap_min
2026-10-28 22:15:25.2606063	2026-10-28 20:45:59.7143672	2026-10-28 22:12:35.5754331	0	
2026-10-29 07:42:23.4514356	2026-10-28 22:12:35.5754331	2026-10-29 07:39:25.5285486	15	0
2026-10-29 08:41:58.6706195	2026-10-29 07:39:25.5285486	2026-10-29 08:39:22.2683441	1,235	0
2026-10-29 09:01:12.9347079	2026-10-29 08:39:22.2683441	2026-10-29 08:58:19.3470803	390	0
2026-10-29 09:36:07.0164949	2026-10-29 08:58:19.3470803	2026-10-29 09:33:21.4738254	215	0

B. Idempotency in Silver.

```
spark.sql('SELECT COUNT(*) AS c, COUNT(DISTINCT event_id) AS d FROM
dbo.silver_gpu_telemetry').show()
```

Expected: `c = d`. Our result was 1615 = 1615, and it stays the same when the Notebook is re-run with no new data.



```
1  spark.sql("SELECT COUNT(*) AS c, COUNT(DISTINCT event_id) AS d FROM dbo.silver_gpu_telemetry").show()
2  spark.sql("""SELECT model_id, hour_start, records
3               FROM dbo.gold_gpu_hourly
4               WHERE hour_start = TIMESTAMP'2026-06-20 06:00:00'
5               ORDER BY model_id""").show()
```

✓ - Command executed in 2 sec 249 ms by ريم محروس سعد الدغيدى on 10/29/2026, 12:25:49 PM — PySpark (Python)

```
+----+----+
|   c|   d|
+----+----+
|1615|1615|
+----+----+
```

C. Gold consistency with the Eventhouse. Compare unique records per type and hour in the Eventhouse with the `records` column in `dbo.gold_gpu_hourly`.

```
gpu_cleaned
| where ts >= datetime(2026-06-20 06:00:00) and ts < datetime(2026-06-20 07:00:00)
| summarize records = count_distinct(event_id) by model_id, hour_start = bin(ts, 1h)
| project model_id, hour_start, records
| order by model_id asc



spark.sql("""
```

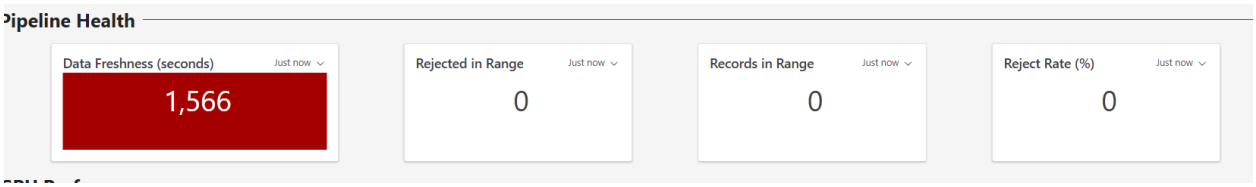
```
SELECT model_id, hour_start, records
FROM dbo.gold_gpu_hourly
WHERE hour_start = TIMESTAMP'2026-06-20 06:00:00'
ORDER BY model_id
""").show()
```

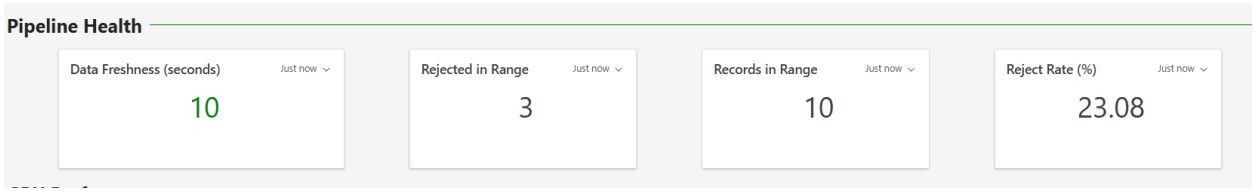
Table 1

model_id	hour_start	records
AMD_MI300X	2026-06-20 06:00:00.000	60
NVIDIA_A100	2026-06-20 06:00:00.000	60
NVIDIA_B200	2026-06-20 06:00:00.000	60
NVIDIA_H100	2026-06-20 06:00:00.000	60
NVIDIA_L40S	2026-06-20 06:00:00.000	60

```
+-----------+-------------------+-------+
|   model_id|         hour_start|records|
+-----------+-------------------+-------+
| AMD_MI300X|2026-06-20 06:00:00|     60|
|NVIDIA_A100|2026-06-20 06:00:00|     60|
|NVIDIA_B200|2026-06-20 06:00:00|     60|
|NVIDIA_H100|2026-06-20 06:00:00|     60|
|NVIDIA_L40S|2026-06-20 06:00:00|     60|
+-----------+-------------------+-------+
```

D. Monitoring. Stopping the Producer for two minutes should raise Data Freshness and drop Arrival Rate to zero. When it restarts, both return to normal.





8. Known Limitations and Future Work

Area	Limitation	Proposed improvement
Cold Path load	Notebook reprocesses the whole table each run	Add `ingest_time` in the Copy step for fully incremental processing
Failure logging	Failed runs are visible in Monitor but not written to `gpu_watermark`	Add a `Log_Failure` activity on the failure path that writes `status = "FAILED"` (not read by the Lookup, so the watermark stays put)
Alerting	Failures are found by looking at dashboards and Monitor	Add alerts on Data Freshness, on pipeline failure, and on `missing > 0`

Area	Limitation	Proposed improvement
Update Policy	Non-transactional, so silent failure is possible	Schedule the reconciliation query as a recurring check
Validation limits	Thresholds are approximate	Review with a domain expert